Subiecte PDSS

Ce tipar de proiectare ar trebui să folosim pentru ca un apel (request) să poată fi procesat de unul dintre mai multe potențiale obiecte handler, și anume de către acela care poate procesa corect apelul (request-ul). Descrieți în câteva fraze elementele distinctive ale tiparului și modul în care acesta ar trebui implementat.

Tiparul potrivit este **Chain of Responsibility**. Ideea: un request este oferit unei succesiuni de obiecte *handler*, fără ca obiectul care a transmis request-ul să știe cine îl va procesa. Fiecare handler decide: dacă poate trata request-ul, îl procesează și se oprește. Dacă nu, îl pasează mai departe către următorul handler din lanț. Implementare: o interfață/clasă abstractă Handler cu metoda handle(Request) și un câmp next. Configuri lanțul la inițializare, apoi clientul apelează primul handler.

Explicați care este diferența între un sistem software rigid și unul fragil. Precizați dacă (și cum!) afectează utilizarea frecventă a specificatorului de acces "public" pentru datele membru ale unei clase rigiditatea și respectiv fragilitatea sistemului software?

Un sistem rigid este greu de modificat: o schimbare mică cere multe schimbări în alte locuri. Un sistem fragil se strică ușor: o schimbare aparent sigură produce defecte neașteptate în părți „neatinse". Date membre publice reduc încapsularea și cresc cuplajul. Astfel, modificarea reprezentării interne a datelor publice va tinde să afecteze direct codul din alte clase. Consecință: crește atât rigiditatea (mai multe dependențe de actualizat), cât și fragilitatea (mai multe locuri unde se pot introduce erori).

La ce se referă relația de agregare între două clase și cum diferă aceasta față de relația de compoziție între clase?

Agregarea este o relație „HAS-A" slabă: un obiect al clasei-întreg („agregatorul") referă obiecte ale altei clase („agregatul"), dar obiectele „agregat" pot exista independent (adică nu sunt construite special pentru acel obiect „agregator") și pot fi partajate (shared). Exemplu: un obiect „Echipă" agregă obiecte „Jucător".

Compoziția este o relație „HAS-A" puternică: obiectele parte („agregatele") aparțin exclusiv obiectului întreg, iar ciclul lor de viață este controlat de acesta. Astfel, dacă obiectul întreg („agregatorul") dispare, dispar și obiectele parte („agregatele"). Exemplu: un obiect „Casă" compune obiecte „Cameră".

Explicați de ce un modul nu poate avea niciodată o "închidere totală" ("100% closure"), în sensul Principiului "Open-Closed". În acest context, explicați și ce tip de închidere ("closure") ar trebui să urmărim, și cum anume putem să realizăm această "închidere"?

Un modul nu poate avea „100% closure” fiindcă apar mereu cerințe noi care nu pot fi prevăzute în avans: schimbări de reguli, interfețe, performanță, securitate etc. Deci vor exista mereu extensii care vor implica modificări ale codului existent. Obiectivul realist este o închidere („closure”) față de variațiile anticipate (axe de schimbare). Acest obiectiv se atinge prin aplicarea celorlalte principii SOLID, mai precis prin dependența de abstracțiuni, construirea modulelor în jurul unei responsabilități unice, și prin segregarea interfeței modulului în funcție de clienții acestuia.

Indicați dacă afirmația de mai jos este adevărată sau falsă și justificați răspunsul (răspunsurile nejustificate sau argumentate greșit fundamental greșit nu se punctează): „Principiul Abstracțiunilor Stabile (Stable Abstractions Principle. SAP) ne spune că pachetele sunt cu atât mai abstracte cu cât sunt mai stabile”. Dacă apreciați că afirmația este adevărată, explicați care este beneficiul aplicării acestui principiu în felul în care a fost el enunțat mai sus, iar dacă apreciați că afirmația e falsă, explicați ce anume e greșit și la ce se referă de fapt principiul.

Afirmația este falsă. SAP nu spune că „stabilitatea implică abstractizare”, ci că pachetele foarte stabile ar trebui să fie și abstracte, iar pachetele instabile pot fi mai concrete. Ideea principiului este următoarea: dacă un pachet are multe dependențe care se bazează pe el (este stabil), modificarea lui e costisitoare. De aceea, pentru ca un pachet foarte stabil (adică unul de care depind multe alte pachete) să permită extensii fără modificări, acesta trebuie să fie cât mai abstract (interfețe, clase abstracte). Elementele de variabilitate se pun în pachete mai instabile. Astfel reducem riscul de modificări în zonele centrale și limităm efectul de propagare a schimbării.

Indicați dacă afirmația de mai jos este adevărată sau falsă și justificați răspunsul (răspunsurile nejustificate sau argumentate greșit fundamental greșit nu se punctează): „Principiul Dependențelor Aciclice (Acyclic Dependencies Principle. ADP) ne spune că între două clase sau între două metode distincte ale unei clase nu ar trebui să existe dependențe ciclice, nici directe, nici indirecte”. Dacă apreciați că afirmația este adevărată, explicați care este beneficiul aplicării acestui principiu în felul în care a fost el enunțat mai sus, iar dacă apreciați că afirmația e falsă, explicați ce anume e greșit și la ce se referă de fapt principiul.

Afirmația este **falsă**. ADP se referă la **dependețe între pachete/module/componente**, nu la cicluri între două clase sau între metode ale aceleiași clase. Principiul spune că graful dependențelor dintre pachete trebuie să fie **aciclic**, altfel apar probleme de build, testare și release (nu poți compila/livra independent. schimbarea se propagă în tot „clusterul” ciclic). Ciclurile între clase sunt și ele de evitat, dar asta ține mai degrabă de design-ul OO (cuplaj), nu de ADP.

Indicați dacă afirmația de mai jos este adevărată sau falsă și justificați răspunsul (răspunsurile nejustificate sau argumentate greșit fundamental greșit nu se punctează): „Principiul Inversării Dependențelor, ne spune că o clasă de bază este un modul de nivel mai înalt față de o clasă derivată, și de aceea sensul dependenței trebuie inversat: clasa derivată trebuind

mereu să fie cea care depinde de clasa de bază". Dacă apreciați că afirmația este adevărată, explicați care este beneficiul aplicării acestui principiu în felul în care a fost el enunțat mai sus, iar dacă apreciați că afirmația e falsă, explicați ce anume e greșit și la ce se referă de fapt principiul.

Afirmația este **falsă**. DIP nu vorbește despre „clasă de bază vs. clasă derivată", ci despre module de nivel înalt (politici) și module de nivel jos (detalii). Principiul spune: (1) modulele de nivel înalt nu trebuie să depindă de module de nivel jos. Ambele trebuie să depindă de abstracțiuni. (2) abstracțiunile nu trebuie să depindă de detalii, ci detaliile de abstracțiuni. În OOP, extensia Derived : Base face deja derivata să depindă de bază. „Inversarea" din DIP înseamnă că și nivelul înalt, și detaliile (implementări concrete) depind de o interfață/clasă abstractă stabilă, iar legarea se face prin injectare (Dependency Injection). Beneficiu: schimbi implementări fără să modifici codul de nivel înalt, deci cuplajul este mai redus și testarea mai ușoară (fiind posibilă înlocuirea dependențelor cu obiecte „mock").

Indicați dacă afirmația de mai jos este adevărată sau falsă și justificați răspunsul (răspunsurile nejustificate sau argumentate greșit fundamental greșit nu se punctează): "Atunci când într-un sistem descoperim dependențe ciclice, modul principal în care se aplică Principiul Dependențelor Aciclice este acela de a folosi tiparul de proiectare Acyclic Visitor". Dacă apreciați că afirmația este adevărată, explicați care este beneficiul aplicării acestui principiu în modul descris mai sus. Iar dacă apreciați că afirmația e falsă, scrieți explicit ce anume e greșit în afirmație.

Afirmația este **falsă**. ADP este un principiu de arhitectură la nivel de pachete/module. Nu spune că soluția „principală" este tiparul Acyclic Visitor. Acyclic Visitor este o tehnică pentru a evita dependențe între ierarhia de visitors și ierarhia de elemente. Pentru ADP, ciclurile se rup prin: reorganizare pachete, extragerea interfețelor într-un pachet mai stabil, aplicarea DIP, introducerea unui pachet comun (facade/abstractions), sau refactorizarea dependențelor astfel încât graful pachetelor să devină aciclic.

Indicați dacă afirmația de mai jos este adevărată sau falsă și justificați răspunsul (răspunsurile nejustificate sau argumentate greșit fundamental greșit nu se punctează): "Relația de agregare între două clase se referă la includerea clasei "parte" în clasa "întreg", în așa fel încât noua clasă va conține toate metodele și obiectele din cele două clase originale." Dacă apreciați că afirmația este adevărată, explicați care este beneficiul agregării. Iar dacă apreciați că afirmația e falsă, scrieți explicit ce anume e greșit în afirmație, și la ce anume se referă agregarea.

Afirmația este **falsă**. Ea descrie mai degrabă „combinarea" a două clase (ca prin moștenire), nu agregarea. **Agregarea** este o relație „HAS-A" în care un obiect al clasei „întreg" **deține o referință** (dată membru) către obiect(e) din clasa „parte". Metodele nu sunt „incluse" automat.

Întregul folosește partea prin delegare. Partea poate exista independent și poate fi partajată între mai multe obiecte „întreg”.

Indicați dacă afirmația de mai jos este adevărată sau falsă și justificați răspunsul (răspunsurile nejustificate nu se punctează): “Diferența față de ‘închiderea totală’ a unui modul este că la ‘închiderea strategică’ mergem pe strategia în care modulul rămâne deschis la extensii și îl ‘închidem’ doar la modificări.” Dacă apreciați că afirmația este adevărată, explicați care este beneficiul închiderii strategice. Iar dacă apreciați că afirmația e falsă, scrieți explicit ce anume e greșit în afirmație, și la ce anume se referă cele două tipuri de “închidere”.

Afirmația este **falsă**. „Deschis la extensii și închis la modificări” este chiar formularea standard a OCP. Nu definește distinct „închidere strategică”. Diferența reală este: închiderea totală ar însemna „nu mai modificăm niciodată modulul pentru nicio cerință”. Ceea ce este imposibil în mod practic. Închiderea strategică înseamnă că „închidem” modulul față de un set de variații anticipate (axe de schimbare), nu față de orice schimbare.

Să presupunem că avem o clasă de bază („Base”) cu o subclasă („Derived”). În clasa de bază există metoda “doSomething()” care apoi e reimplementată în clasa derivată. Codul din cele două metode “doSomething()” este aproape identic. Mai puțin o linie care e diferită (să zicem că în clasa de bază se apelează o metodă „x()” și în cea derivată se apelează o metodă „y()”). Ce tipar de proiectare am putea folosi pentru a rezolva problema. Prezentați o soluție în pseudo-cod (sau într-un limbaj de programare orientat-pe-obiecte) care să ilustreze implementarea tiparului. Nu se depunctează nici un fel de greșeli sintactice, atâta vreme cât implementarea reflectă corect mecanismele orientate-pe-obiecte (ex. moștenire, polimorfism).

Tiparul potrivit: **Template Method** (cu „hook method”). Ideea tiparului este următoarea: pui algoritmul comun în Base.doSomething() și extragi linia variabilă într-o metodă protejată, suprascrisă în Derived.

```
abstract class Base {
  public final void doSomething() {
    step1();
    variableStep();   // aici era x() vs y()
    step3();
  }

  protected void step1() { /* cod comun */ }
  protected void step3() { /* cod comun */ }

  protected abstract void variableStep(); // hook
}
```

```
class Derived extends Base {
  protected void variableStep() { y(); }
  private void y() { /* specific */ }
}

class ConcreteBase extends Base {
  protected void variableStep() { x(); }
  private void x() { /* specific */ }
}
```

Indicați dacă afirmația de mai jos este adevărată sau falsă și justificați răspunsul (răspunsurile nejustificate sau argumentate greșit fundamental greșit nu se punctează): În mod ideal ierarhiile de conținere (containment) între clase/obiecte ar trebui să fie organizate "în lățime", adică să fie cât mai late și cât mai puțin adânci, pentru că astfel clasa "rădăcină" (cea aflată în vârful ierarhiei de conținere, și care deci direct sau indirect conține instanțele altor clase) este mai ușor de utilizat de către clienți.

Afirmația este **falsă**. O ierarhie de conținere „cât mai lată" face clasa „rădăcină" să cunoască și să gestioneze multe tipuri de părți, deci crește responsabilitățile și cuplajul acelei clase întreg față de clasele parte. Afirmația mai are însă și o a doua greșeală chiar și mai importantă: clienții clasei rădăcină nu ar trebuie să știe niciodată structura internă a acestei clase. Din perspectiva clasei client, obiectivul nu este „lat vs. adânc", ci: responsabilități clare, încapsulare, dependențe minime și navigare limitată (ex. Law of Demeter).

Indicați dacă afirmația de mai jos este adevărată sau falsă și justificați răspunsul (răspunsurile nejustificate sau argumentate greșit fundamental greșit nu se punctează): „Principiul Segregării Interfeței (Interface Segregation Principle) se referă la faptul că fiecare clasă ar trebui să implementeze un număr limitat de interfețe (nu mai mult de 6)"

Afirmația este **falsă**. ISP nu impune niciun prag numeric (6 sau altul). Principiul spune că un client nu trebuie forțat să depindă de metode pe care nu le folosește, deci e mai bine să ai interfețe mici, coezive, decât o interfață „grasă" folosită de mulți clienți diferiți. O clasă poate implementa multe interfețe dacă acestea sunt bine segregate și reflectă roluri distincte.

Indicați dacă afirmația de mai jos este adevărată sau falsă și justificați răspunsul (răspunsurile nejustificate sau argumentate greșit fundamental greșit nu se punctează): „La principiul Inversării Dependențelor, inversiunea se referă la inversarea relației de moștenire în cazul unei subclase B care nu suprascrie corect, din punct de vedere design by contract, o metodă din superclasa A. Pe scurt, în acest caz în loc ca B să extindă pe A, A ar trebui să extindă pe B."

Afirmația este **falsă**. DIP nu are legătură cu „inversarea moștenirii” A←B și nici nu este un remediu pentru încălcarea Design by Contract la suprascriere. Problema descrisă în enunț ține de Liskov Substitution Principle: B trebuie să respecte contractul lui A (nu întărește precondiții, nu slăbește postcondiții, menține invarianții). DIP spune că modulele de nivel înalt nu trebuie să depindă de module de nivel mai jos, ci ambele trebuie să depindă de abstracțiuni.

Ce spune Principiul Dependențelor Stabile și respectiv Principiul Abstracțiunilor Stabile? Care este legătura între cele două?

Principiul Dependențelor Stabile (SDP) spune că dependențele dintre pachete trebuie să fie orientate înspre pachete mai stabile. Astfel, un pachet nu ar trebui să depindă de un pachet mai instabil decât sine. Principiul Abstracțiunilor Stabile (SAP) spune că pachetele stabile ar trebui să fie cât mai abstracte (interfețe, clase abstracte), ca să poată fi extinse fără modificări. Legătura: SDP împinge arhitectura spre un nucleu stabil, iar SAP spune cum să proiectăm acel nucleu. Și anume să îl facem abstract ca să nu devină rigid.

Explicați la ce se referă criteriul de modularitate al „Continuității”. De ce utilizarea frecventă de cod duplicat (copy-paste-adapt) este un semn că acest criteriu de modularitate (al „Continuității”) nu este îndeplinit?

Criteriul „Continuității” cere ca o schimbare mică în cerințe să producă o schimbare mică, localizată în cod. Dacă apare mult copy-paste-adapt, aceeași regulă există în mai multe locuri. O modificare a regulii trebuie repetată în toate copiile. Schimbarea devine mai mare, dispersată și ușor de omis. Deci continuitatea este încălcată, iar sistemul devine mai rigid și mai fragil.

Se poate vorbi despre aplicarea „Design by Contract” atunci când o metodă dintr-o clasă apelează o metodă dintr-o altă clasă oarecare, adică dintr-o clasă cu care nu se află într-o relație de moștenire? Justificați răspunsul.

Da. Design by Contract se aplică între orice funcție client (apelant) și funcție furnizor/provider (metoda apelată), indiferent de moștenire. Contractul este definit prin precondițiile, postcondițiile și invarianții clasei furnizor/provider. Clientul trebuie să asigure precondițiile înainte de apel. Funcția apelată garantează postcondițiile după execuție și menține invarianții. Moștenirea devine relevantă doar când contractele trebuie păstrate la suprascriere (LSP).